

Communications inter-agents du modèle SCONTO-SVU

Diagrammes UML d'activités et de séquence
fondés sur le modèle générique validé

Modèle audité : `model/SCONTO_SVU_GENERIC_MASTER.alp`
Dépôt : `sconto-svu-generic-anylogic-model`, branche
`docs/uml-agent-communications`

Document technique de présentation, destiné à un lecteur ne connaissant pas nécessairement le
code AnyLogic sous-jacent.

Table des matières

1	Objectif et périmètre	2
1.1	Sources de vérité utilisées	2
1.2	Écart de nommage identifié entre les sources, corrigé dans ce document	2
2	Légende générale	2
2.1	Légende des agents	3
2.2	Lecture des communications AER	3
2.3	Flux informationnels et flux physiques	3
2.4	Statuts de validation utilisés	3
3	Architecture globale des agents	4
4	Flux global	5
5	Tronc commun de réception commande	7
6	Cas MTS	9
6.1	Cas de stock insuffisant : voir Source/Make/Deliver	10
7	Source / Make / Deliver	11
8	Réapprovisionnement autonome	13
9	Séquences d'interaction	14
9.1	SEQ-01 — Réception et validation d'une commande client	14
9.2	SEQ-02 — Livraison directe depuis le stock MTS	14
9.3	SEQ-03 — Insuffisance matière : consultation fournisseur	17
9.4	SEQ-04 — Approvisionnement et retour vers Make	20
9.5	SEQ-05 — Planification et exécution Make	23
9.6	SEQ-06 — Livraison finale, réception client et reporting	24
10	AER et remontée hiérarchique	24
11	Couverture actuelle des stratégies MTS, MTO et ETO	24
12	Tableau synthétique des communications	25
13	Limites et statut de validation	29
14	Conclusion	29

1 Objectif et périmètre

Ce document présente, sous forme de diagrammes UML d'activités et de séquence, les communications réellement implémentées entre les agents du modèle générique SCOTO-SVU. Il s'adresse à un lecteur qui doit comprendre le fonctionnement du système sans lire le code AnyLogic lui-même.

Aucun diagramme de ce document n'a été construit à partir de connaissances générales sur SCOR ou sur les systèmes multi-agents. Chaque message, chaque agent et chaque séquence provient d'une vérification directe dans le fichier modèle ou dans une documentation déjà audité et confrontée à un export runtime. Lorsqu'un élément n'a pas pu être confirmé, il est explicitement signalé comme tel plutôt que représenté comme acquis.

Ce document ne modifie aucun fichier .alp, aucun code Java, aucun scénario JSON et ne touche ni à la logique du modèle ni aux travaux de consolidation E4/E5 en cours sur d'autres branches.

1.1 Sources de vérité utilisées

Par ordre de priorité, conformément à la méthode de cette passe :

1. `model/SCOTO_SVU_GENERIC_MASTER.alp` (fonction `tracerFluxHierarchique()`, seule fonction du modèle qui journalise et anime les communications AER ; 81 sites d'appel recensés, 69 messages distincts) ;
2. `POST_MERGE_COMPLETENESS_AUDIT.md` (inventaire des agents, architecture AER, statut MTS/MTO/ETO) ;
3. `EXPERIMENTATION_FOUNDATION.md` (preuves runtime du Run 01, seed 1001) ;
4. l'annexe déjà publiée *Catalogue des messages inter-agents et Flux global* du dépôt de documentation (`documentation/appendices/g-messages-flux-global.tex`), elle-même construite par confrontation d'un document de travail générique avec `sources/model/SCOTO_SVU_FINAL_VALIDATED.alp` et l'ABox d'un run de validation antérieur (`RUN_1773129600000_1788264883846` : un run distinct du Run 01 d'EXP-0, cité ici comme seconde preuve runtime indépendante, pas comme le même run) ;
5. les traces et exports du Run 01 (EXP-0, seed 1001) pour la confirmation MTS/Source/Make/Deliver/réapprovisionnement autonome.

Un document de travail intitulé « **Tableaux clarifiés du flux global** » a été identifié comme source conceptuelle de l'annexe ci-dessus, elle-même explicitement citée par un commentaire du code actuel (fonction `traiterCommande()` : « *Tronc commun strict C2 → C6 du document Flux global* »). Ce document a servi de point de départ conceptuel, confronté ici à l'implémentation réelle de `model/SCOTO_SVU_GENERIC_MASTER.alp` : la quasi-totalité des messages qu'il recense a été retrouvée telle quelle par recherche indépendante dans le fichier modèle actuel, confirmant que l'architecture de communication décrite est restée stable entre les deux fichiers.

1.2 Écart de nommage identifié entre les sources, corrigé dans ce document

L'annexe citée ci-dessus utilise le préfixe `ASup-s...` pour le rôle de pilotage opérationnel (ex. `ASup-sD1`). La fonction `idSuperviseurCommande()` du `model/SCOTO_SVU_GENERIC_MASTER.alp` actuel produit désormais le préfixe `AOp-s...` pour ce même rôle (ex. `AOp-sD1.1`). Il s'agit d'une évolution de nommage entre les deux fichiers, pas d'une erreur : ce document utilise systématiquement `AOp-s...`, conforme au modèle actuellement audité, et le signale explicitement ici plutôt que de laisser une incohérence silencieuse entre les sources.

2 Légende générale

2.1 Légende des agents

Identifiant	Rôle
AS	Strategic Agent : pilotage stratégique global, PI/SCOR consolidé (une instance unique, AS-sP1).
AT	Tactical Agent : arbitrage tactique par macro-processus SCOR (AT-sP2 Source, AT-sP3 Make, AT-sP4 Deliver).
CA	Coordinator Agent : coordination par macro-processus et par stratégie (CA-sS1, CA-sM1, CA-sD1).
AOp	Operational Pilot Agent : pilotage opérationnel d'un poste/groupe (ex. AOp-sM1.1). Nommé ASup dans une documentation antérieure (voir écart signalé ci-dessus).
A0e	Operational Execution Agent : exécution opérationnelle sur une micro-activité précise (ex. A0e-sD1.8).
SupplierActor	Acteur fournisseur externe.
CustomerActor	Acteur client externe.

Dans le modèle, AOp/A0e/CA/AT ne sont pas des classes séparées par rôle : `OperationalAgent` est une classe unique dont l'identifiant (AOp-... ou A0e-...) encode le rôle et l'activité concernée. Cette distinction reste utile à la lecture des diagrammes, même si elle n'implique pas de classe Java distincte (confirmé dans `POST_MERGE_COMPLETENESS_AUDIT.md`, §2.3).

2.2 Lecture des communications AER

- **AMENDMENT** (bleu, trait plein) : instruction, plan, ordre ou demande descendant dans l'organisation.
- **EXECUTION** (vert-bleu, trait plein) : réalisation ou événement opérationnel constaté.
- **REPORT** (gris, trait pointillé) : retour d'information vers les niveaux de pilotage.

Cette classification est reprise directement de l'usage du modèle (premier paramètre de `tracerFluxHierarchique(phase, émetteur, destinataire, message, ...)`), pas d'une théorie AER générale.

2.3 Flux informationnels et flux physiques

Les échanges entre agents (les messages ci-dessus) sont des **flux informationnels** : ordre, requête, réponse, rapport, notification. Un second type de flux, **physique** (brun, trait pointillé épais avec triangle), représente le déplacement réel de matière ou de produit (fournisseur → entreprise, produit fini → client). Dans les diagrammes de séquence, ce flux physique est annoté explicitement plutôt que représenté comme un message UML ordinaire.

2.4 Statuts de validation utilisés

Statut	Signification
Observé en runtime	Confirmé par une trace ABox d'un run réellement exécuté (Run 01 seed 1001, ou le run de validation antérieur cité en annexe).
Implémenté dans le modèle	Confirmé par lecture du code (<code>tracerFluxHierarchique</code>), sans confirmation par une trace runtime disponible.
Implémenté mais non validé en runtime	Le mécanisme existe dans le code mais aucun run documenté ne l'a exercé (ex. MTO/ETO).
Partiel / legacy	Le mécanisme existe mais est inactif, incomplet, ou coexiste avec une variante plus récente.

Le mot « validé » n'est jamais utilisé seul dans ce document lorsque la preuve disponible est uniquement statique.

3 Architecture globale des agents

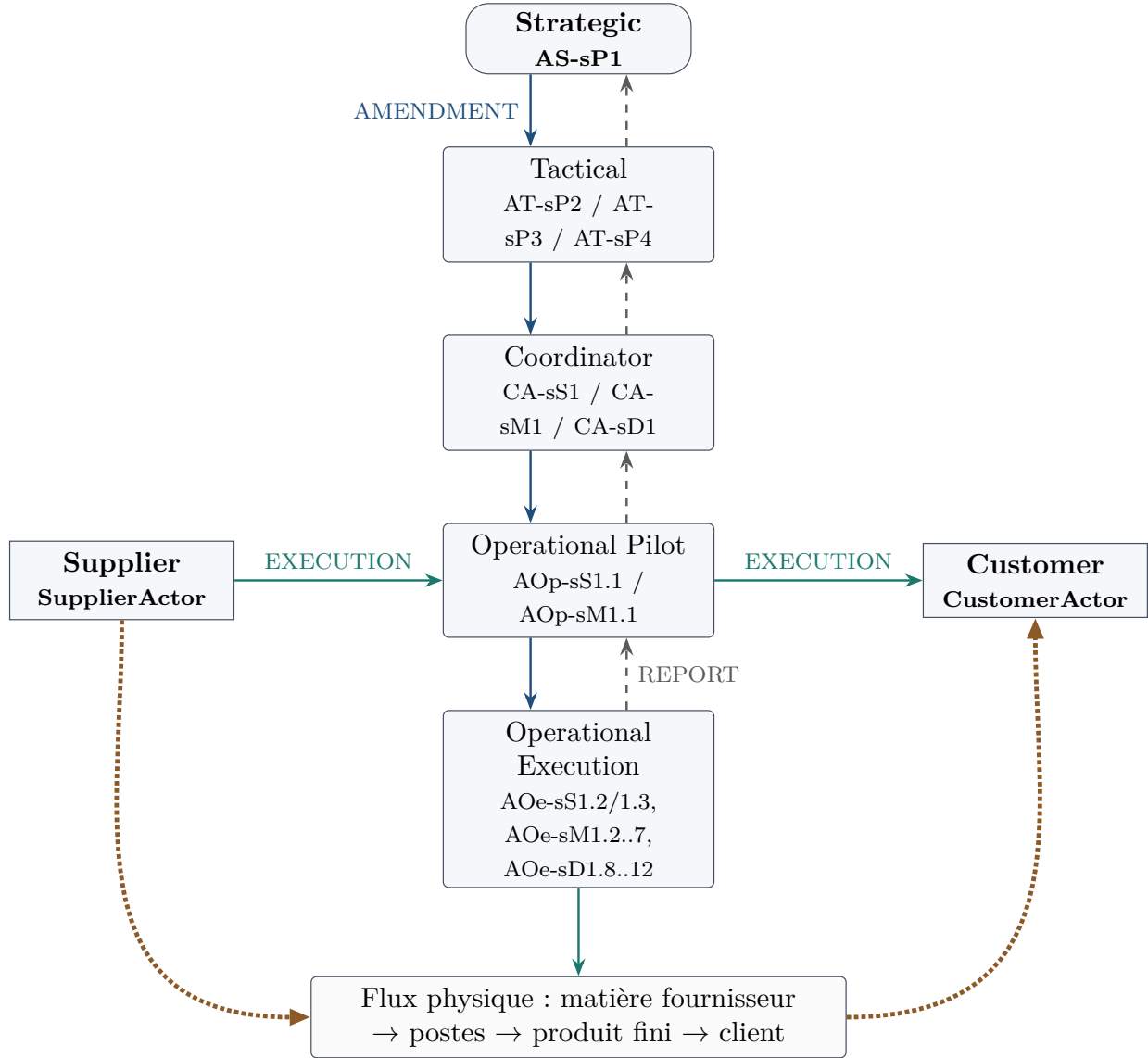


FIGURE 1 – Architecture globale des communications multi-agents SCOTO-SVU.

Objectif. Donner une vue d'ensemble de la hiérarchie de pilotage avant les cas détaillés : comment une instruction descend (**AMENDMENT**), comment un événement opérationnel remonte (**REPORT/EXECUTION**), et où se situent les acteurs externes (fournisseur, client) et le flux physique.

Légende. Une seule instance conceptuelle par niveau est représentée ; le modèle réel instancie plusieurs **CoordinatorAgent/OperationalAgent**, un par macro-processus SCOR (Source, Make, Deliver), non multipliés ici par lisibilité (voir §12, tableau synthétique, pour le détail par macro-processus).

Interprétation. Le flux descendant (**AMENDMENT**) part de l'agent stratégique unique et se spécialise à chaque niveau jusqu'à l'exécution opérationnelle, qui seule interagit directement avec

le fournisseur et le client. Le flux remontant (REPORT) suit rigoureusement le chemin inverse, sans saut de niveau observé dans le code. Le flux physique (matière, produit fini) est distinct des communications d'agents : il relie directement les acteurs externes à l'exécution opérationnelle.

Statut : Implémenté dans le modèle (architecture confirmée par lecture exhaustive des identifiants d'agents et des fonctions de génération d'identifiants ; la chaîne complète AMENDMENT descendante et REPORT remontante est observée en runtime pour le cas MTS, Run 01).

4 Flux global

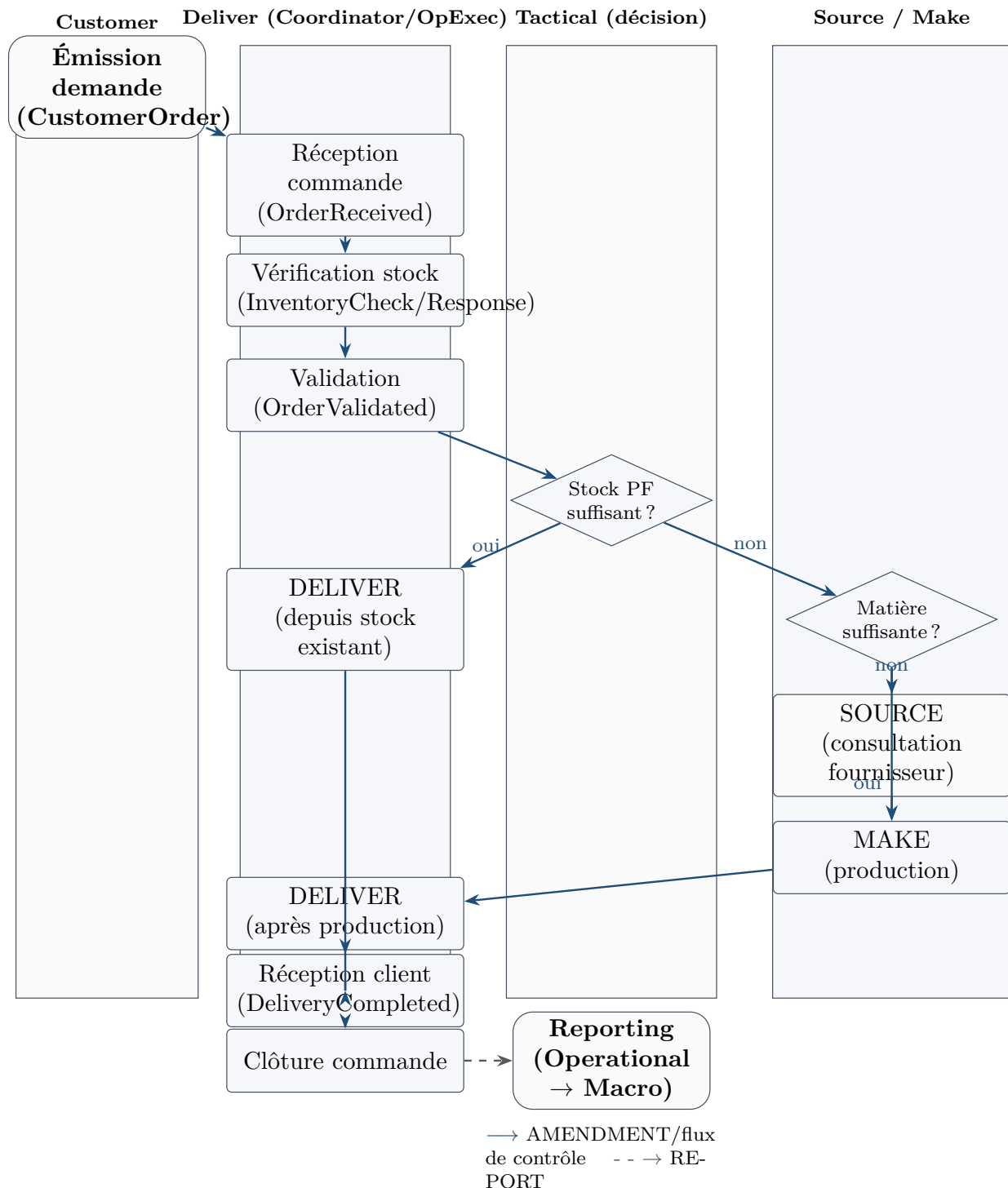


FIGURE 2 – ACT-01 — Flux métier global, du besoin client à la clôture et au reporting.

Objectif. Montrer, en un seul diagramme d'activité à couloirs, l'ensemble du chemin possible d'une commande : réception, décision, branchement Source/Make selon l'état du stock, Deliver, clôture et reporting.

Légende. Couloirs : Customer, Deliver (Coordinator/Exécution), Tactical (décision), Source/Make. Les deux losanges de décision correspondent exactement aux deux tests réellement présents dans le code (`analyserStockCommandeOrchestree()` : stock produit fini suffisant ; `analyserMatiereCommandeOrchestree()` : matière suffisante).

Interprétation. Une seule commande peut suivre trois chemins distincts selon l'état du stock et de la matière au moment de son traitement. Le branchement n'est jamais un choix du client : c'est une décision interne calculée à partir de l'état réel du système.

Statut : Implémenté dans le modèle ; le chemin « stock PF suffisant » est observé en runtime (Run 01) ; les deux chemins passant par Source/Make sont implémentés mais non exercés par le Run 01 (voir §14).

5 Tronc commun de réception commande

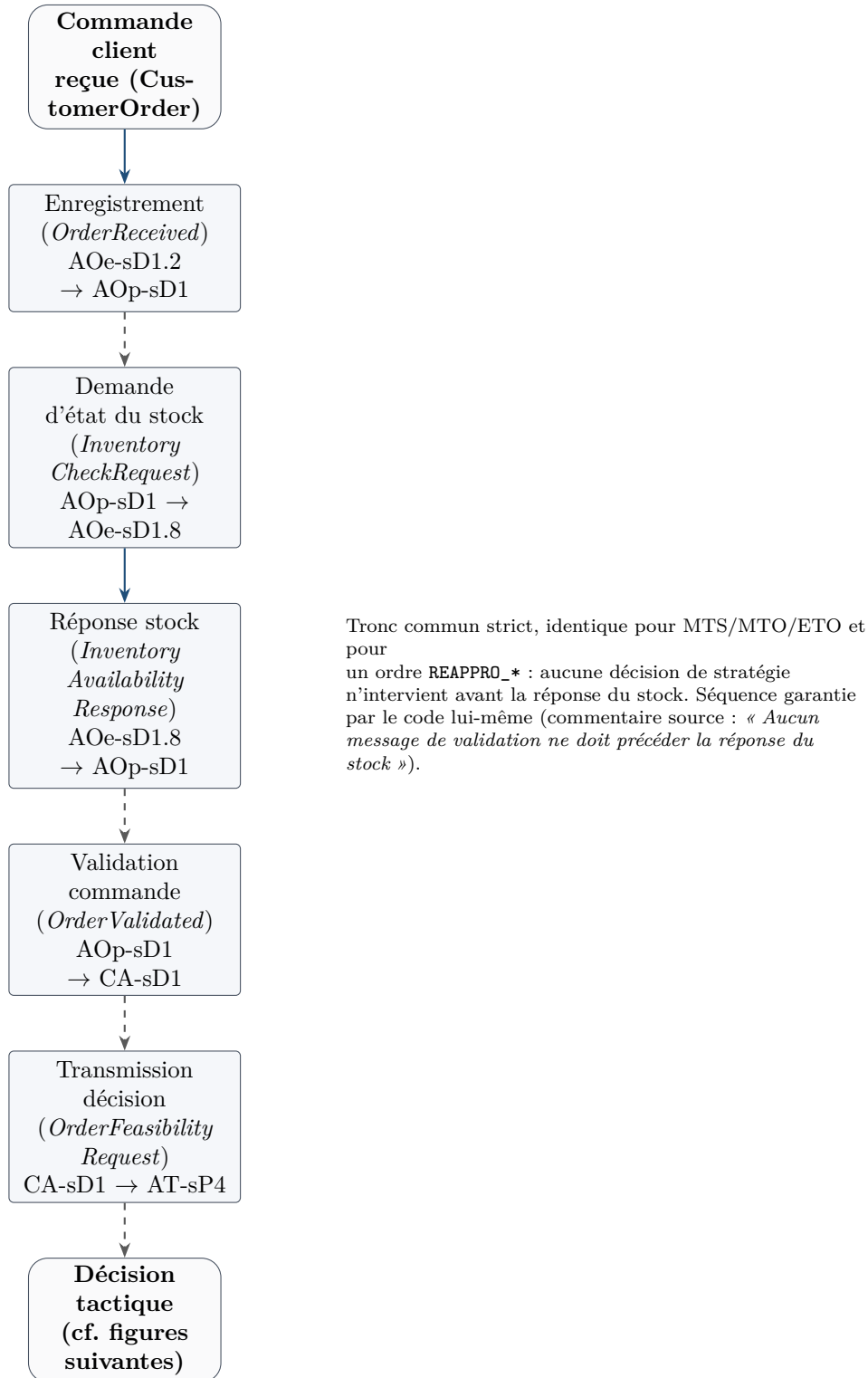


FIGURE 3 – ACT-02 — Tronc commun A : réception et validation de la commande.

Objectif. Détailler la portion strictement commune à MTS, MTO, ETO et aux ordres REAPPRO_* : aucune décision de stratégie n'intervient avant que le système dispose de l'état du stock.

Légende. Chaque flèche porte le nom du message réellement journalisé par le code (`tracerFlux Hierarchique`), avec l'émetteur et le destinataire réels.

Interprétation. Le code impose explicitement cet ordre (commentaire source cité en §2.1) : `OrderReceived` $\rightarrow$ `InventoryCheckRequest` $\rightarrow$ `InventoryAvailabilityResponse` $\rightarrow$ `OrderValidated` $\rightarrow$ `OrderFeasibilityRequest`. C'est seulement après ce tronc que la décision de stratégie (figures suivantes) peut avoir lieu.

Statut : Observé en runtime (Run 01 et run de validation antérieur cité en annexe).

6 Cas MTS

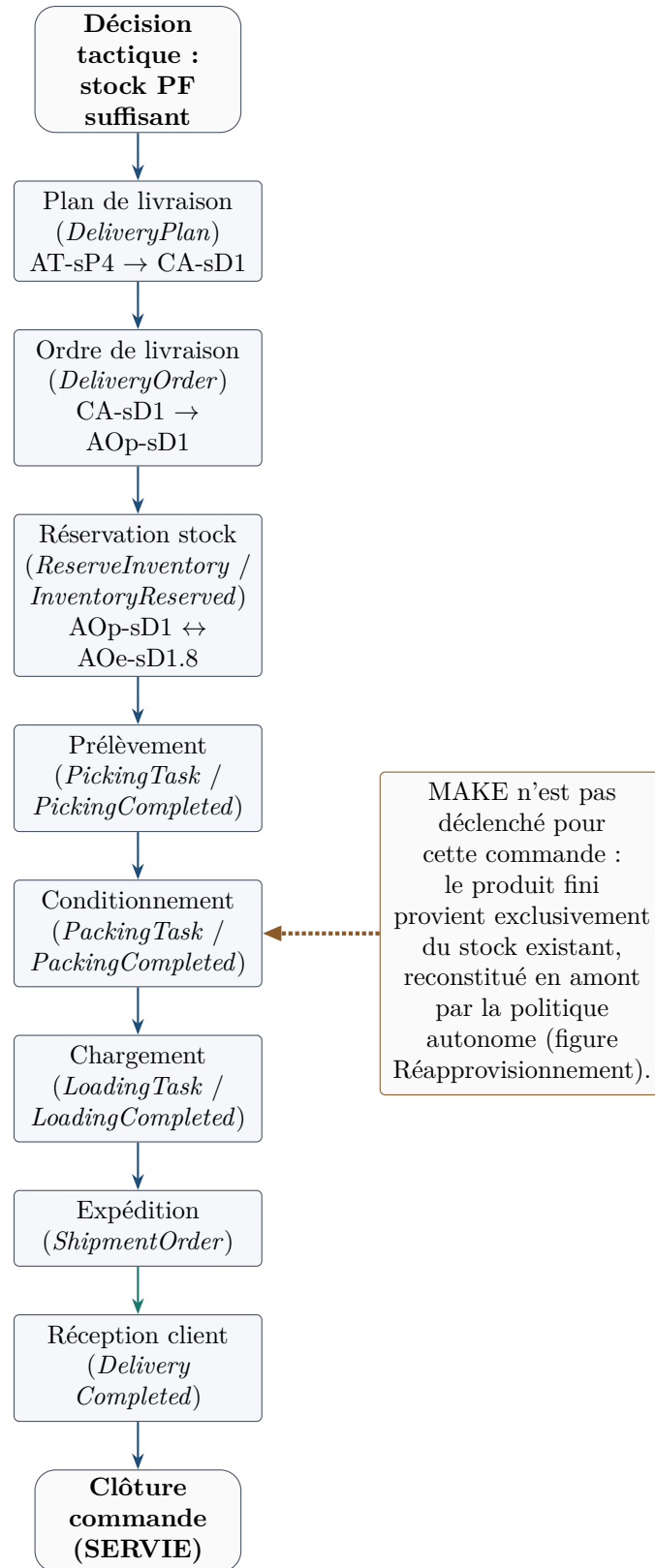


FIGURE 4 – ACT-03 — Cas MTS : stock produit fini suffisant, livraison directe.

Objectif. Représenter le chemin observé en runtime lorsqu’une commande MTS trouve un stock produit fini suffisant.

Légende. Chaque étape correspond à un message réellement journalisé, de `DeliveryPlan` à `DeliveryCompleted`.

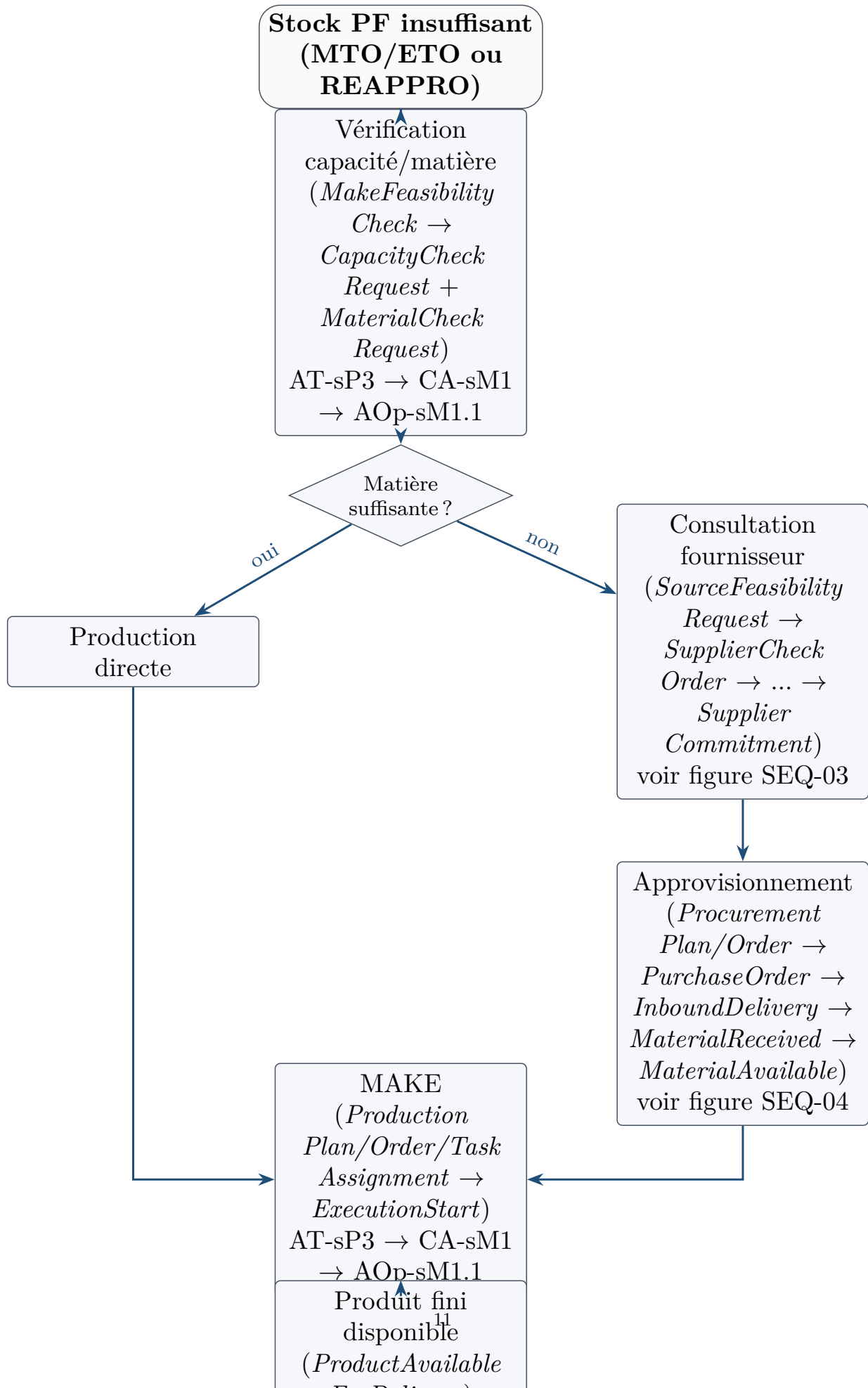
Interprétation. Ce chemin ne déclenche jamais `Make` : le produit fini provient exclusivement d'un stock déjà constitué par la politique autonome (§9). C'est le seul chemin complet observé de bout en bout par le Run 01.

Statut : Observé en runtime (Run 01, seed 1001).

6.1 Cas de stock insuffisant : voir `Source/Make/Deliver`

Le cas « stock PF insuffisant mais matière disponible » et le cas « stock PF et matière insuffisants » ne sont pas deux diagrammes séparés dans ce document : le code les traite comme deux branches d'une même décision (§7), et aucun des deux n'a été exercé par un run documenté à ce jour. Les présenter comme des diagrammes indépendants aurait suggéré une preuve runtime qui n'existe pas.

7 Source / Make / Deliver



Objectif. Montrer le point de décision réel du code (`analyserMatiereCommandeOrchestree()`) et les deux branches qu'il ouvre.

Légende. Le losange reprend exactement le test `matieresSuffisantesPourScenario()` du code. Les identifiants d'agents renvoient aux séquences détaillées SEQ-03 et SEQ-04.

Interprétation. MTO et ETO empruntent rigoureusement le même chemin dans ce diagramme : le code ne distingue pas les deux stratégies au-delà de leur classification SCOR (voir §12 « Couverture MTS/MTO/ETO »). Un ordre `REAPPRO_*` emprunte également ce même point d'entrée (§9).

Statut : Implémenté dans le modèle ; aucune des deux branches (matière suffisante ou insuffisante) n'a été exercée par le Run 01, qui n'a mobilisé que le chemin MTS direct. Non validé en runtime dans la campagne actuelle.

8 Réapprovisionnement autonome

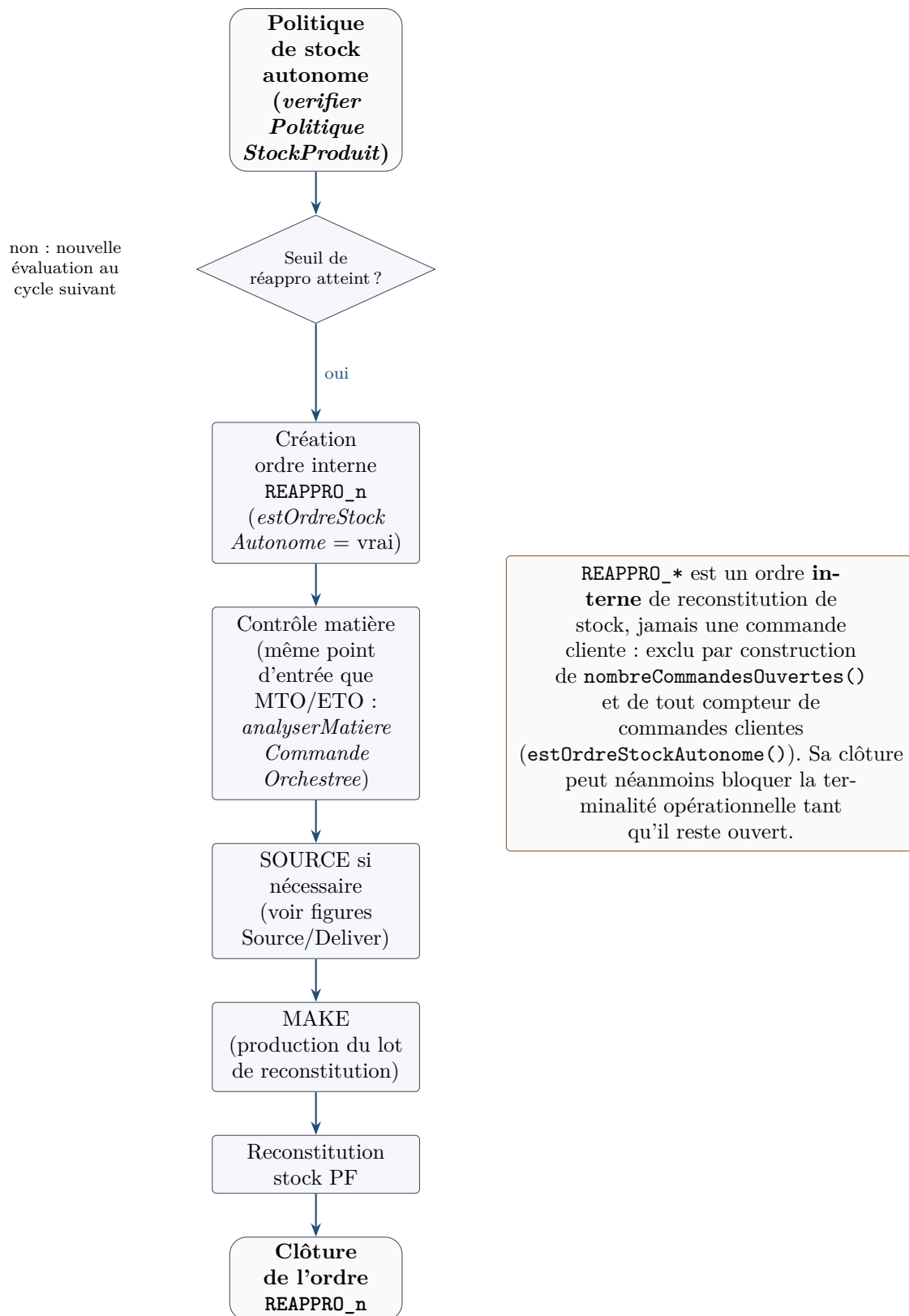


FIGURE 6 – ACT-06 — Réapprovisionnement autonome (ordre interne REAPPRO_n).

Objectif. Montrer que la reconstitution du stock produit fini est pilotée par une politique autonome, indépendante des commandes clientes, mais empruntant les mêmes points d'entrée

Source/Make.

Légende. Le losange correspond au test de seuil de la politique de stock (`verifierPolitiqueStockProduit()`).

Interprétation. Un ordre `REAPPRO_*` n'est jamais une commande cliente : il est explicitement exclu de tout compteur de commandes clientes (`estOrdreStockAutonome()`), mais sa clôture conditionne la terminalité opérationnelle globale au même titre qu'une commande cliente ouverte.

Statut : Observé en runtime (Run 01 : ordre `REAPPRO_1`, clôturé avant la fin du run, unité finale `REAPPRO_1_F_50`).

9 Séquences d'interaction

9.1 SEQ-01 — Réception et validation d'une commande client

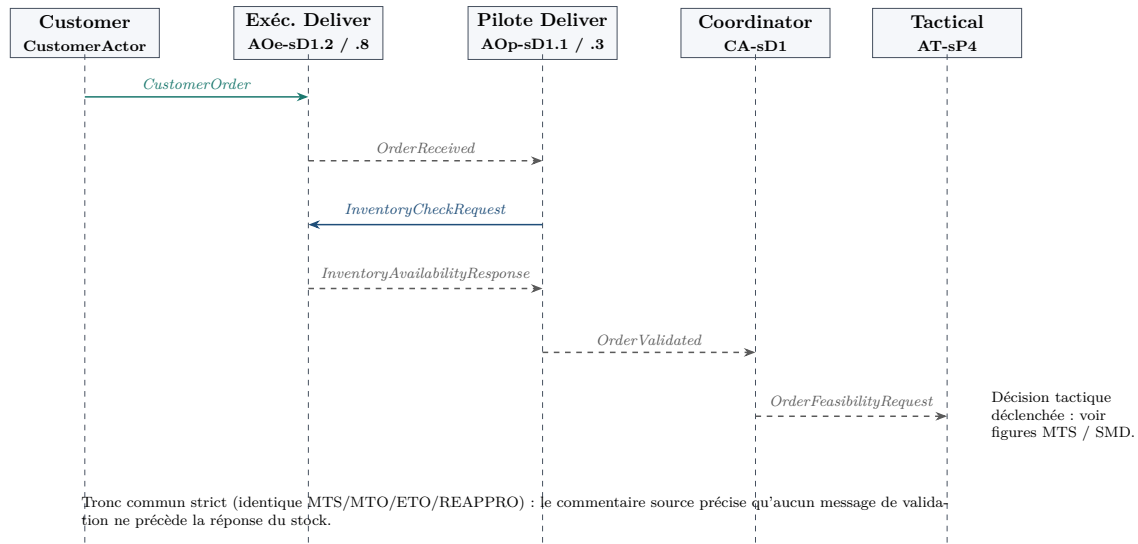


FIGURE 7 – SEQ-01 — Réception et validation d'une commande client.

Interprétation. Le client déclenche le flux (`CustomerOrder`) ; l'information descend et remonte strictement entre les niveaux d'exécution, de pilotage et de coordination du macro-processus Deliver, avant qu'une décision de stratégie ne soit demandée au niveau tactique.

Statut : Observé en runtime.

9.2 SEQ-02 — Livraison directe depuis le stock MTS

15

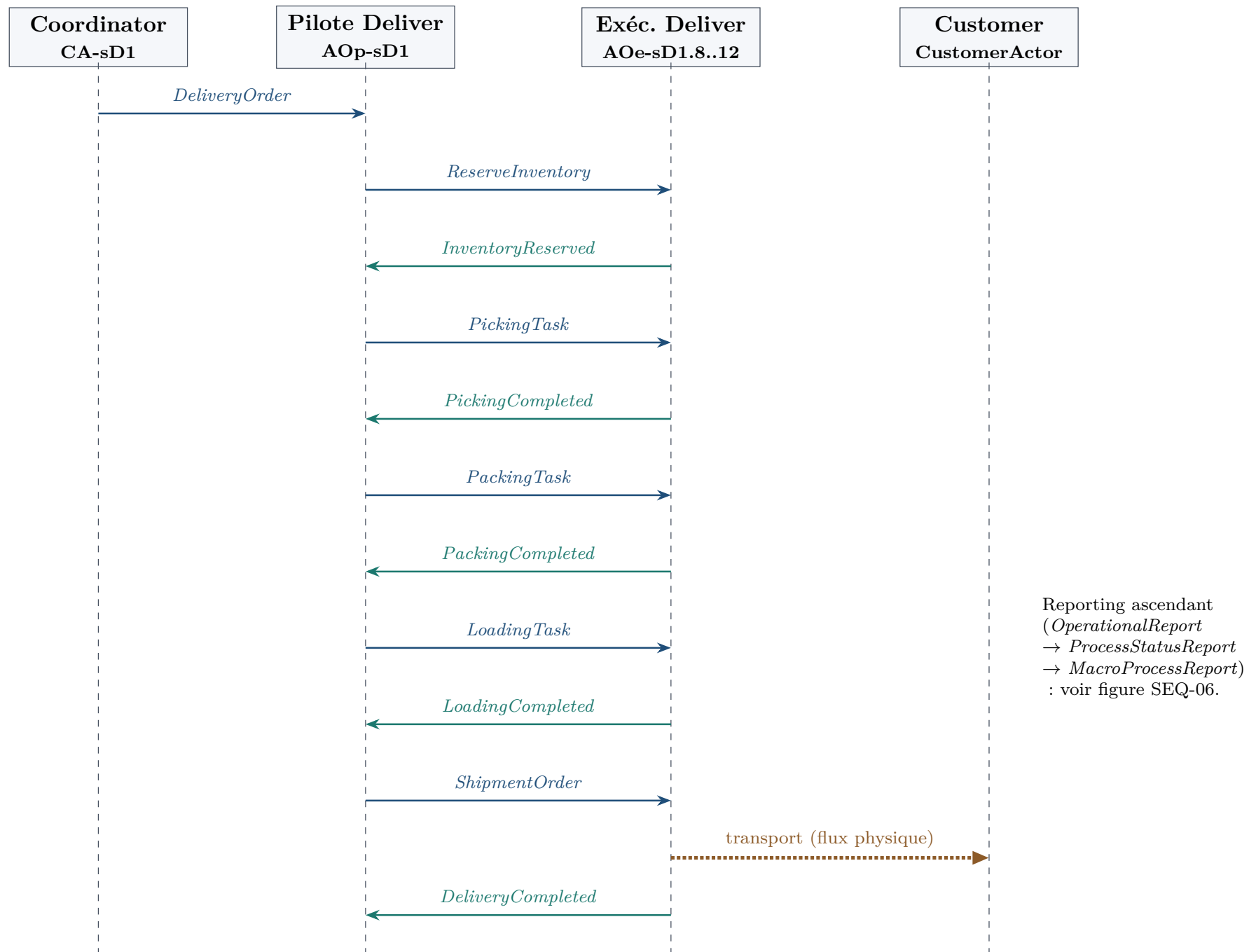


FIGURE 8 – SEQ-02 — Livraison directe depuis le stock MTS.

Interprétation. Chaque étape physique (prélèvement, conditionnement, chargement) est encadrée par un message d'instruction (AMENDMENT) et un message de constat (EXECUTION), avant le transport physique réel vers le client.

Statut : Observé en runtime (Run 01).

9.3 SEQ-03 — Insuffisance matière : consultation fournisseur

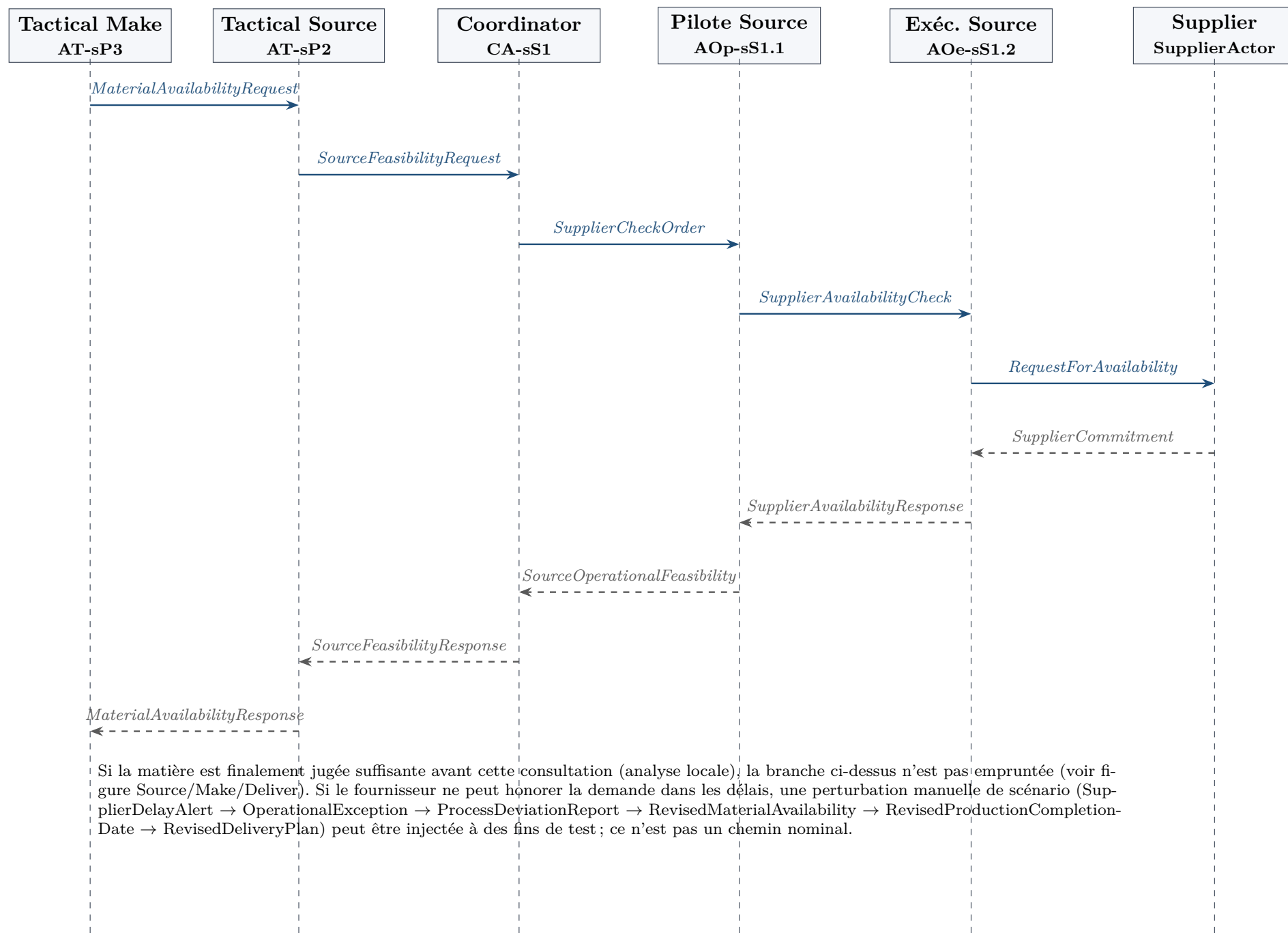


FIGURE 9 — SEQ-03 — Consultation fournisseur en cas de matière insuffisante

Interprétation. La demande descend du tactique Make vers le tactique Source, puis jusqu'au fournisseur externe, avant que la réponse ne remonte symétriquement. La perturbation de retard fournisseur (note de la figure) est un mécanisme de test manuel, pas un chemin nominal.

Statut : Implémenté mais non validé en runtime dans la campagne actuelle (aucun run documenté n'a emprunté cette branche).

9.4 SEQ-04 — Approvisionnement et retour vers Make

22

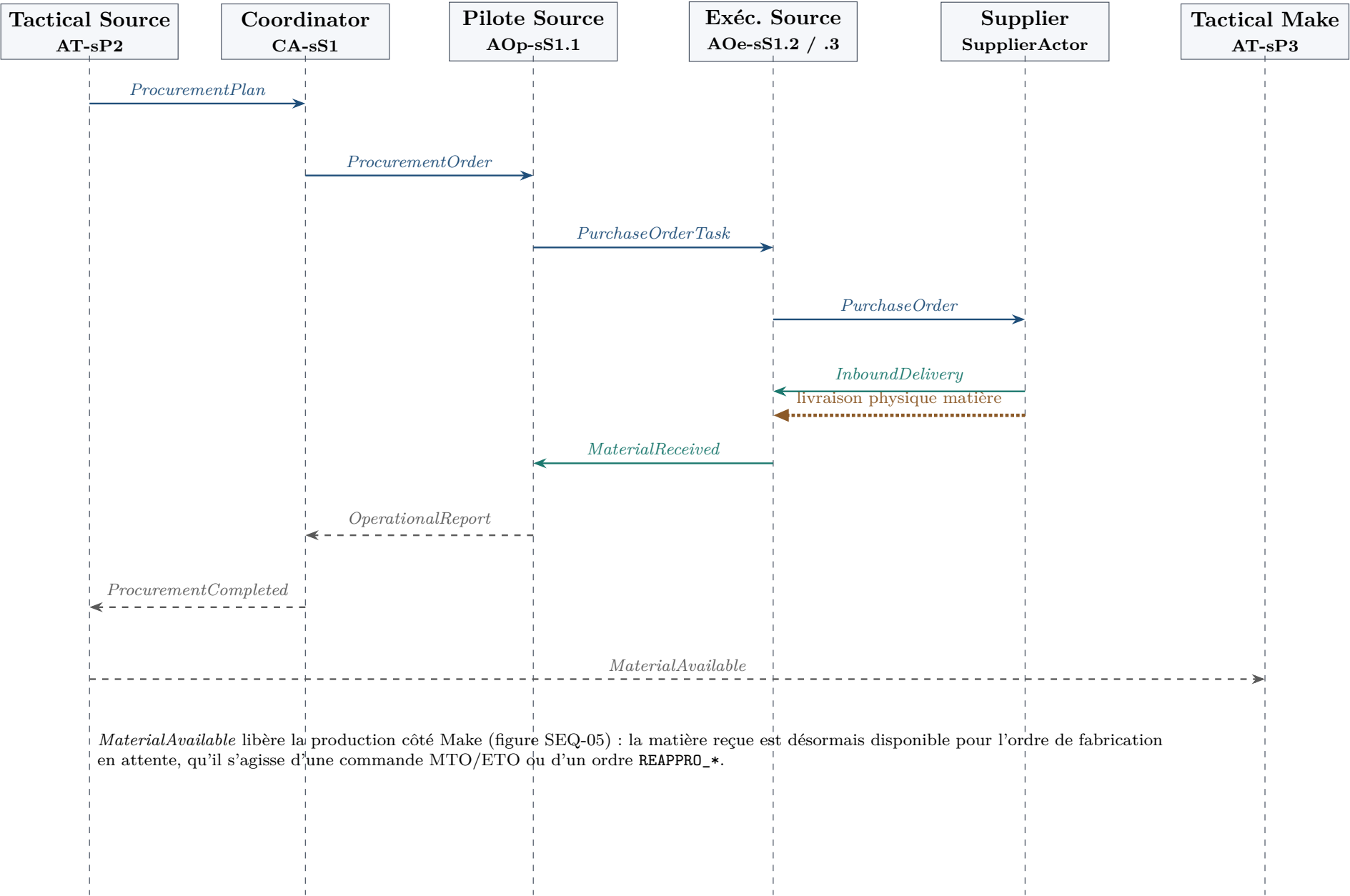


FIGURE 10 – SEQ-04 — Approvisionnement matière et retour vers Make.

Interprétation. L’approvisionnement se conclut par un message `MaterialAvailable` transmis directement au tactique Make, qui peut alors lancer la production (SEQ-05).

Statut : Implémenté mais non validé en runtime dans la campagne actuelle.

9.5 SEQ-05 — Planification et exécution Make

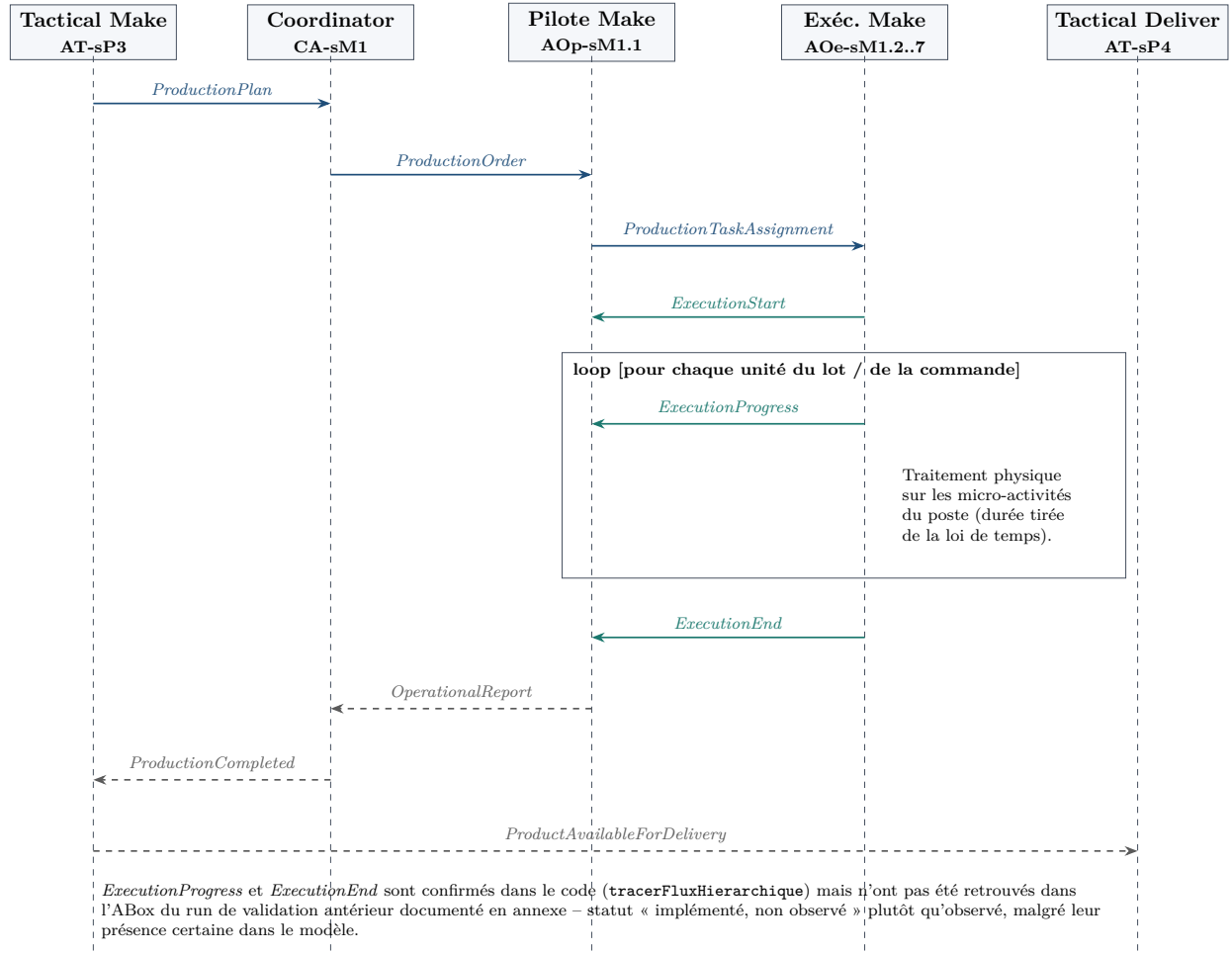


FIGURE 11 – SEQ-05 — Planification et exécution de la production.

Interprétation. Le fragment *loop* représente le traitement physique répété des unités du lot sur les micro-activités du poste, sans représenter individuellement chaque unité produite (contrairement aux 50 unités effectivement tracées dans le Run 01 pour l’ordre REAPPRO_1).

Statut : Partiellement observé : la planification (`ProductionPlan/Order/TaskAssignment`) et `ExecutionStart` sont confirmés par le code ; `ExecutionProgress/ExecutionEnd` sont implémentés mais non retrouvés dans l’ABox du run de validation cité en annexe — statut « implémenté, non observé ».

9.6 SEQ-06 — Livraison finale, réception client et reporting

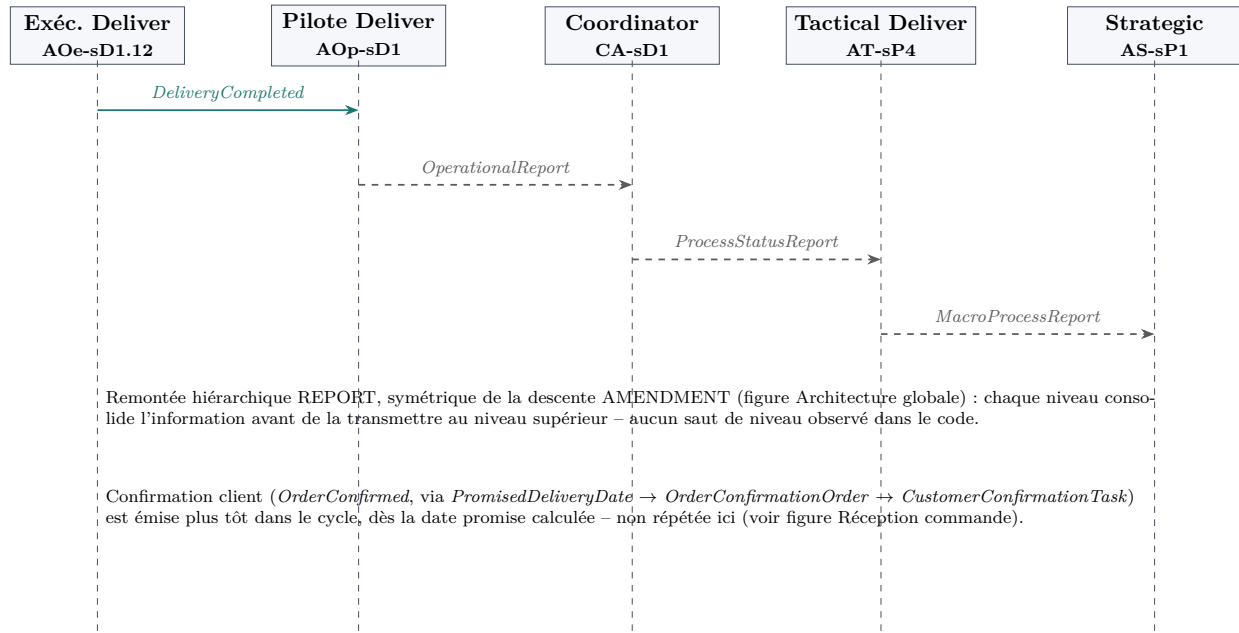


FIGURE 12 – SEQ-06 — Clôture de la livraison et remontée du reporting.

Interprétation. La remontée REPORT est strictement symétrique de la descente AMENDMENT de l'architecture globale (§4) : `OperationalReport` → `ProcessStatusReport` → `MacroProcessReport`, jusqu'à l'agent stratégique unique.

Statut : Observé en runtime (Run 01).

10 AER et remontée hiérarchique

Les trois catégories de messages déjà présentées en §3.2 structurent l'ensemble des figures de ce document : **AMENDMENT** porte la décision et l'ordre vers le bas, **EXECUTION** porte le constat opérationnel, **REPORT** porte la consolidation vers le haut. Cette classification n'est pas une théorie ajoutée après coup : c'est le premier paramètre littéral de la fonction `tracerFluxHierarchique()` à chacun de ses 81 sites d'appel dans le modèle, et elle correspond au type ontologique porté par chaque message dans l'export ABox (`AmendmentMessage`, `ExecutionMessage/ExecutionStartMessage`, `ReportingMessage`, confirmé par l'annexe citée en §2.1).

11 Couverture actuelle des stratégies MTS, MTO et ETO

Ce document ne construit pas trois architectures différentes pour MTS, MTO et ETO, parce que le modèle lui-même n'en construit pas trois.

- **MTS** est observé en runtime (Run 01) et détaillé dans les figures ACT-03/SEQ-02.
- **MTO** est implémenté : une commande MTO emprunte exactement le même point d'entrée que la branche « stock insuffisant » (§8, ACT-04/05), et le code confirme qu'aucune branche MTO propre n'existe au-delà de ce point commun avec ETO. **Non validé en runtime** dans la campagne actuelle.
- **ETO** est implémenté via le même traitement partagé que MTO. Aucune phase d'ingénierie ou de personnalisation propre à ETO n'est représentée dans le pipeline de traitement de commande (confirmé par `POST_MERGE_COMPLETENESS_AUDIT.md`, §3) : la distinction ETO existe uniquement au niveau de la classification SCOR (niveau 3, groupes tactiques

CA-sS3/CA-sM3/CA-sD3) et d'une constante de délai, pas dans le flux de messages présenté ici. **Non validé en runtime**, et sans diagramme de séquence propre pour cette raison.

Créer des diagrammes distincts pour MTO et ETO aurait suggéré une différence de comportement que le code ne présente pas à ce niveau de détail.

12 Tableau synthétique des communications

TABLE 1 – Communications inter-agents représentées dans ce document (échantillon complet des figures ACT-02 à SEQ-06 ; catalogue complet en annexe).

Émetteur	Récepteur	Message	AER	SCOR	Cas	Validation
CustomerActor	AOe-sD1.2	CustomerOrder	EXECUTION	Deliver	Réception mande	com- Observé runtime
AOe-sD1.2	AOp-sD1.1	OrderReceived	REPORT	Deliver	Réception mande	com- Observé runtime
AOp-sD1.3	AOe-sD1.8	InventoryCheckRequest	AMENDMENT	Deliver	Réception mande	com- Observé runtime
AOe-sD1.8	AOp-sD1.3	InventoryAvailabilityResponse	REPORT	Deliver	Réception mande	com- Observé runtime
AOp-sD1.1	CA-sD1	OrderValidated	REPORT	Deliver	Réception mande	com- Observé runtime
CA-sD1	AT-sP4	OrderFeasibilityRequest	REPORT	Plan/ Deliver	Réception mande	com- Observé runtime
AT-sP4	CA-sD1	DeliveryPlan	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
CA-sD1	AOp-sD1	DeliveryOrder	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
AOp-sD1	AOe-sD1.8	ReserveInventory	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
AOe-sD1.8	AOp-sD1	InventoryReserved	EXECUTION	Deliver	MTS stock suffisant	Observé runtime
AOp-sD1	AOe-sD1.9	PickingTask	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
AOe-sD1.9	AOp-sD1	PickingCompleted	EXECUTION	Deliver	MTS stock suffisant	Observé runtime
AOp-sD1	AOe-sD1.10	PackingTask	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
AOe-sD1.10	AOp-sD1	PackingCompleted	EXECUTION	Deliver	MTS stock suffisant	Observé runtime
AOp-sD1	AOe-sD1.11	LoadingTask	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
AOe-sD1.11	AOp-sD1	LoadingCompleted	EXECUTION	Deliver	MTS stock suffisant	Observé runtime
AOp-sD1	AOe-sD1.12	ShipmentOrder	AMENDMENT	Deliver	MTS stock suffisant	Observé runtime
AOe-sD1.12	AOp-sD1	DeliveryCompleted	EXECUTION	Deliver	MTS stock suffisant	Observé runtime
AOp-sD1	CA-sD1	OperationalReport	REPORT	Deliver	Livraison/reporting	Observé runtime
CA-sD1	AT-sP4	ProcessStatusReport	REPORT	Deliver	Livraison/reporting	Observé runtime
AT-sP4	AS-sP1	MacroProcessReport	REPORT	Deliver	Livraison/reporting	Observé runtime
AT-sP3	CA-sM1	MakeFeasibilityCheck	AMENDMENT	Make	Source/Make/ Deliver	Implémenté, non validé
CA-sM1	AOp-sM1.1	CapacityAndMaterialCheckOrder	AMENDMENT	Make	Source/Make/ Deliver	Implémenté, non validé
AOp-sM1.1	AOe-sM1.2..7	CapacityCheckRequest / MaterialCheckRequest	AMENDMENT	Make	Source/Make/ Deliver	Implémenté, non validé
AOe-sM1.2..7	AOp-sM1.1	CapacityAvailabilityResponse / MaterialAvailabilityResponse	REPORT	Make	Source/Make/ Deliver	Implémenté, non validé
CA-sM1	AT-sP3	MakeFeasibilityResponse	REPORT	Make	Source/Make/ Deliver	Implémenté, non validé
AT-sP3	AT-sP2	MaterialAvailabilityRequest	AMENDMENT	Source	Consultation	four-nisseur Implémenté, non validé
AT-sP2	CA-sS1	SourceFeasibilityRequest	AMENDMENT	Source	Consultation	four-nisseur Implémenté, non validé
CA-sS1	AOp-sS1.1	SupplierCheckOrder	AMENDMENT	Source	Consultation	four-nisseur Implémenté, non validé

Émetteur	Récepteur	Message	AER	SCOR	Cas	Validation
AOp-sS1.1	AOe-sS1.2	SupplierAvailabilityCheck	AMENDMENT	Source	Consultation nisseur	four- Implémenté, non validé
AOe-sS1.2	SupplierActor	RequestForAvailability	AMENDMENT	Source	Consultation nisseur	four- Implémenté, non validé
SupplierActor	AOe-sS1.2	SupplierCommitment	REPORT	Source	Consultation nisseur	four- Implémenté, non validé
AOe-sS1.2	AOp-sS1.1	SupplierAvailabilityResponse	REPORT	Source	Consultation nisseur	four- Implémenté, non validé
AOp-sS1.1	CA-sS1	SourceOperationalFeasibility	REPORT	Source	Consultation nisseur	four- Implémenté, non validé
CA-sS1	AT-sP2	SourceFeasibilityResponse	REPORT	Source	Consultation nisseur	four- Implémenté, non validé
AT-sP2	AT-sP3	MaterialAvailabilityResponse	REPORT	Source	Consultation nisseur	four- Implémenté, non validé
AT-sP2	CA-sS1	ProcurementPlan	AMENDMENT	Source	Approvisionnement	Implémenté, non validé
CA-sS1	AOp-sS1.1	ProcurementOrder	AMENDMENT	Source	Approvisionnement	Implémenté, non validé
AOp-sS1.1	AOe-sS1.2	PurchaseOrderTask	AMENDMENT	Source	Approvisionnement	Implémenté, non validé
AOe-sS1.2	SupplierActor	PurchaseOrder	AMENDMENT	Source	Approvisionnement	Implémenté, non validé
SupplierActor	AOe-sS1.3	InboundDelivery	EXECUTION	Source	Approvisionnement (flux physique)	Implémenté, non validé
AOe-sS1.3	AOp-sS1.1	MaterialReceived	EXECUTION	Source	Approvisionnement	Implémenté, non validé
AOp-sS1.1	CA-sS1	OperationalReport	REPORT	Source	Approvisionnement	Implémenté, non validé
CA-sS1	AT-sP2	ProcurementCompleted	REPORT	Source	Approvisionnement	Implémenté, non validé
AT-sP2	AT-sP3	MaterialAvailable	REPORT	Source	Approvisionnement	Implémenté, non validé
AT-sP3	CA-sM1	ProductionPlan	AMENDMENT	Make	Planification	Make Implémenté, non validé
CA-sM1	AOp-sM1.1	ProductionOrder	AMENDMENT	Make	Planification	Make Implémenté, non validé
AOp-sM1.1	AOe-sM1.2..7	ProductionTaskAssignment	AMENDMENT	Make	Planification	Make Implémenté, non validé
AOe-sM1.2..7	AOp-sM1.1	ExecutionStart	EXECUTION	Make	Planification	Make Implémenté, non validé
AOe-sM1.2..7	AOp-sM1.1	ExecutionProgress / ExecutionEnd	EXECUTION	Make	Planification	Make Implémenté, non observé
AOp-sM1.1	CA-sM1	OperationalReport	REPORT	Make	Planification	Make Implémenté, non validé
CA-sM1	AT-sP3	ProductionCompleted	REPORT	Make	Planification	Make Implémenté, non observé

Émetteur	Récepteur	Message	AER	SCOR	Cas	Validation
AT-sP3	AT-sP4	ProductAvailableForDelivery	REPORT	Make	Planification Make	Implémenté, non observé
SupplierActor	AOp-sS1.1	SupplierDelayAlert	REPORT	Source	Perturbation (test manuel)	Implémenté, non validé en nominal
AOp-sS1.1	CA-sS1	OperationalException	REPORT	Source	Perturbation (test manuel)	Implémenté, non validé en nominal
CA-sS1	AT-sP2	ProcessDeviationReport	REPORT	Source	Perturbation (test manuel)	Implémenté, non validé en nominal
AT-sP2	AT-sP3	RevisedMaterialAvailability	REPORT	Source	Perturbation (test manuel)	Implémenté, non validé en nominal
AT-sP3	AT-sP4	RevisedProductionCompletionDate	REPORT	Make	Perturbation (test manuel)	Implémenté, non validé en nominal
AT-sP4	CA-sD1	RevisedDeliveryPlan	AMENDMENT	Deliver	Perturbation (test manuel)	Implémenté, non validé en nominal

Ce tableau reprend les communications effectivement représentées dans les figures de ce document. Le catalogue complet des 62 messages avec leur statut détaillé (`OBSERVE_RUN_FINAL/IMPLEMENTE_NON_OBSERVE`) reste publié dans l'annexe déjà citée du dépôt de documentation (§2.1) et n'est pas republié intégralement ici pour éviter la duplication d'une source déjà auditée.

13 Limites et statut de validation

Élément	Niveau de preuve exact
MTS	Observé durant le Run 01 (seed 1001) et durant le run de validation antérieur cité en annexe.
Source	Observé durant le run de validation antérieur (annexe); observé indirectement durant le Run 01 lors de la reconstitution de stock <code>REAPPRO_1</code> (le chemin exact emprunté — consultation fournisseur ou matière déjà suffisante — n'a pas été isolé spécifiquement dans cette passe).
Make	Observé durant le Run 01 et le run de validation antérieur.
Deliver	Observé durant le Run 01 et le run de validation antérieur.
Réapprovisionnement autonome	Observé durant le Run 01 (ordre <code>REAPPRO_1</code> , clôturé).
MTO	Implémenté (code confirmé), non validé en runtime dans la campagne actuelle.
ETO	Implémenté via le traitement partagé avec MTO; aucune phase d'ingénierie distincte n'est validée, ni même formalisée séparément dans le pipeline de traitement de commande.
Return	Un identifiant de coordinateur dédié existe dans le code (<code>idCoordinateurCommande("R", ...) → CA-sR</code>), mais aucun message AER de la famille Return n'a été retrouvé parmi les 81 sites d'appel de <code>tracerFluxHierarchique()</code> recensés, ni dans l'annexe déjà auditée contre un run réel. Return reste donc disponible dans l'architecture (identifiant réservé) sans qu'un flux de messages propre soit implémenté ou observé.
Multiproduit	Non présenté comme validé en runtime dans ce document : le Run 01 comme le run de validation antérieur portent sur un scénario mono-produit. Voir <code>POST_MERGE_COMPLETENESS_AUDIT.md</code> §17 pour l'audit structurel complet.

Aucune interaction représentée dans ce document ne provient d'une supposition : chaque message porte un nom retrouvé littéralement dans `model/SCONTO_SVU_GENERIC_MASTER.alp` ou dans l'annexe déjà auditée contre un run réel.

14 Conclusion

Le modèle générique SCONTO-SVU implémente une architecture de communication multi-agents cohérente et cataloguée : trois catégories AER (AMENDMENT, EXECUTION, REPORT), cinq niveaux de pilotage (Strategic, Tactical, Coordinator, Operational Pilot, Operational Execution), et un vocabulaire de messages stable entre deux fichiers modèles distincts dans le temps (l'annexe déjà auditée et le master générique actuel). Le chemin MTS complet, de la réception de commande à la clôture et au reporting, est observé en runtime et documenté ici avec le niveau de détail nécessaire pour qu'un lecteur non familier du code AnyLogic comprenne le mécanisme. Les chemins MTO, ETO et la consultation fournisseur restent implémentés mais non exercés par un run documenté à ce jour ; ce document ne les présente jamais comme validés, conformément à la exigence de traçabilité qui structure l'ensemble de cette fondation expérimentale SCONTO-SVU.